1 Linear Discriminant Analysis (LDA)

[1]:
```
import numpy as np
```

1.1 1. Define the classes

[2]:
```
class_1 = np.array([(4, 1), (2, 4), (2, 3), (3, 6), (4, 4)], dtype=float)
class_2 = np.array([(9, 10), (6, 8), (9, 5), (8, 7), (10, 8)], dtype=float)

print("Class 1:\n", class_1)
print("\nClass 2:\n", class_2)
print("\nSample counts:", len(class_1), len(class_2))
```

```
Class 1:
 [[4. 1.]
 [2. 4.]
 [2. 3.]
 [3. 6.]
 [4. 4.]]

Class 2:
 [[ 9. 10.]
 [ 6.  8.]
 [ 9.  5.]
 [ 8.  7.]
 [10.  8.]]

Sample counts: 5 5
```

1.2 2. Calculate the class means

[3]:
```
# Mean of each class
mean_1 = class_1.mean(axis=0)
mean_2 = class_2.mean(axis=0)

print("Mean of class 1:", mean_1)
print("Mean of class 2:", mean_2)
```

```
Mean of class 1: [3.  3.6]
Mean of class 2: [8.4 7.6]
```

1.3 3. Calculate within-class scatter

[4]:
```
# Differences from each class mean
centered_1 = class_1 - mean_1
centered_2 = class_2 - mean_2

print("Class 1 deviations:\n", centered_1)
print("\nClass 2 deviations:\n", centered_2)
```

```
Class 1 deviations:
 [[ 1.  -2.6]
```

Name: Swapnaraj Mohanty
SIC No: 24BCSH93
Lab Roll No: 11
Date: 2nd November 2026

```
 [-1.   0.4]
 [-1.  -0.6]
 [ 0.   2.4]
 [ 1.   0.4]]

Class 2 deviations:
 [[ 0.6  2.4]
 [-2.4  0.4]
 [ 0.6 -2.6]
 [-0.4 -0.6]
 [ 1.6  0.4]]
```

[5]:
```
# Average scatter matrix of each class, as in the numerical example
scatter_1 = centered_1.T @ centered_1 / len(class_1)
scatter_2 = centered_2.T @ centered_2 / len(class_2)

print("S1:\n", scatter_1)
print("\nS2:\n", scatter_2)
```

```
S1:
 [[ 0.8  -0.4 ]
 [-0.4   2.64]]

S2:
 [[ 1.84 -0.04]
 [-0.04  2.64]]
```

[6]:
```
# Sum the class scatter matrices
within_class_scatter = scatter_1 + scatter_2
print("Within-class scatter (SW):\n", within_class_scatter)
```

```
Within-class scatter (SW):
 [[ 2.64 -0.44]
 [-0.44  5.28]]
```

1.4 4. Calculate between-class scatter

[7]:
```
# Difference between the class means
mean_difference = mean_1 - mean_2
between_class_scatter = np.outer(mean_difference, mean_difference)

print("Mean difference:", mean_difference)
print("\nBetween-class scatter (SB):\n", between_class_scatter)
```

```
Mean difference: [-5.4 -4. ]

Between-class scatter (SB):
 [[29.16 21.6 ]
 [21.6  16.  ]]
```

Name: Swapnaraj Mohanty
SIC No: 24BCSH93
Lab Roll No: 11
Date: 2nd November 2026

1.5 5. Form the LDA matrix

[8]:
```
# M = SW^(-1) SB
lda_matrix = np.linalg.inv(within_class_scatter) @ between_class_scatter
print("LDA matrix (M):\n", lda_matrix)
```

```
LDA matrix (M):
 [[11.89244558  8.80921895]
 [ 5.08194622  3.76440461]]
```

1.6 6. Find the largest eigenvalue and its eigenvector

[9]:
```
# Eigenvalues and corresponding eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(lda_matrix)

print("Eigenvalues:\n", eigenvalues)
print("\nEigenvectors (columns):\n", eigenvectors)
```

```
Eigenvalues:
 [15.65685019+0.j  0.        +0.j]

Eigenvectors (columns):
 [[ 0.91955932+0.j -0.59522755+0.j]
 [ 0.39295122+0.j  0.80355719+0.j]]
```

[10]:
```
# Select and normalize the eigenvector for the largest eigenvalue
largest_index = np.argmax(eigenvalues.real)
largest_eigenvalue = eigenvalues[largest_index].real
projection_vector = eigenvectors[:, largest_index].real
projection_vector /= np.linalg.norm(projection_vector)

print("Largest eigenvalue:", largest_eigenvalue)
print("LDA projection vector:", projection_vector)
```

```
Largest eigenvalue: 15.65685019206146
LDA projection vector: [0.91955932 0.39295122]
```

1.7 7. Verify using the direct formula

[11]:
```
# Direct solution from the class means
direct_vector = np.linalg.solve(within_class_scatter, mean_difference)
direct_vector /= np.linalg.norm(direct_vector)

print("Direct projection vector:", direct_vector)
print("Absolute cosine similarity:", abs(np.dot(projection_vector, direct_vector)))
```

```
Direct projection vector: [-0.91955932 -0.39295122]
Absolute cosine similarity: 1.0
```

Name: Swapnaraj Mohanty
SIC No: 24BCSH93
Lab Roll No: 11
Date: 2nd November 2026

1.8 8. Project the samples onto the LDA direction

[12]:
```
# Project each sample using y = v^T x
projected_class_1 = class_1 @ projection_vector
projected_class_2 = class_2 @ projection_vector

print("Projected class 1:\n", projected_class_1)
print("\nProjected class 2:\n", projected_class_2)
```

```
Projected class 1:
 [4.07118849 3.41092352 3.0179723  5.11638527 5.25004215]

Projected class 2:
 [12.20554606  8.66096567 10.24078996 10.10713308 12.33920294]
```

[13]:
```
# Compare the projected class means
projected_mean_1 = projected_class_1.mean()
projected_mean_2 = projected_class_2.mean()

print("Projected mean of class 1:", projected_mean_1)
print("Projected mean of class 2:", projected_mean_2)
print("Separation between projected means:", abs(projected_mean_1 - projected_mean_2))
```

```
Projected mean of class 1: 4.173302345082105
Projected mean of class 2: 10.710727540529707
Separation between projected means: 6.537425195447602
```

Name: Swapnaraj Mohanty
SIC No: 24BCSH93
Lab Roll No: 11
Date: 2nd November 2026